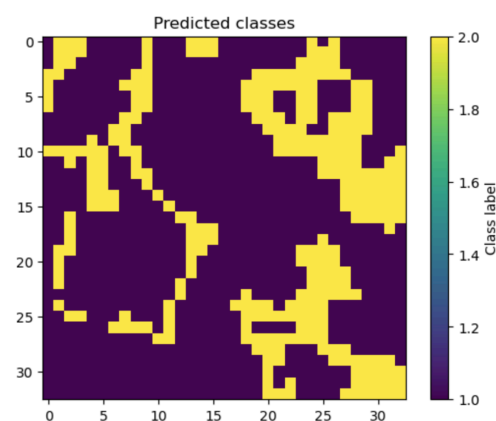


Q3 report:

To find the optimal learning rate t , I implemented a for loop with the function slightly adjusted (I lowered the max iterations to 1000 so that the function can compute quickly for all values of t). Having run this a few times and looking at the graph plotted, I estimated that $t=0.25$ or $t=0.5$ were the best learning rates, as other values either took too long to converge (t too small), or ended up overstepping and oscillating around the minimum as the learning rate was too large. I proceeded to test the convergence and classification with these learning rates on the data with max iterations now set to 100000. I found that $t=0.5$ was quicker to compute, however depending on the randomised initial Θ_0 , sometimes led to inaccurate results, putting all the data in class 1. Hence I prioritised the increased reliability of the classification from $t=0.25$, despite it being marginally slower.

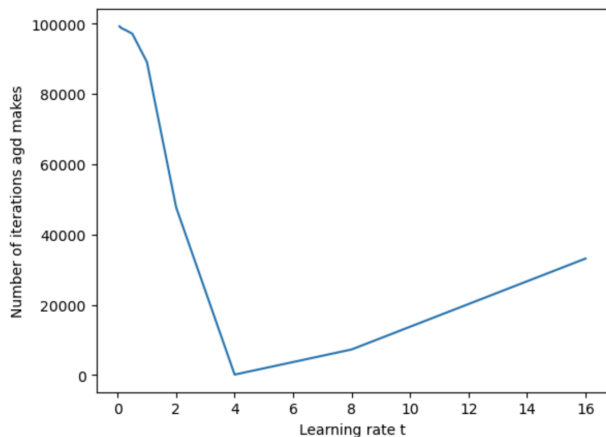
I also did a sanity check on our training data, to check our function was working as intended and convergence with $t=0.25$ gave accurate predictions. I got that all the training data was correctly classified, which was strong evidence that 0.25 was a good choice of t .

Our convergence is weighted significantly towards the first two classes. The function does not seem to classify any data points into the third class. This indicates that our predictions are not entirely accurate. This could potentially be because the features of compound C are significantly smaller valued, meaning they get outweighed by A and B, or potentially the undersampling of the data has removed features relevant to C and not A and B.



Q5 report:

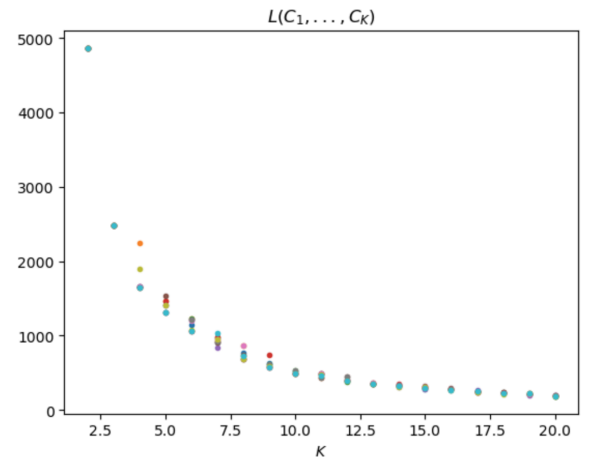
In Q5 I implemented the Kmeans clustering algorithm from sklearn. To decide the optimal number of clusters, I plotted a graph of losses of the clusters using code from problem sheet 4. I chose $k=10$ as the best number, as there was little variation between trials, and there was a slight elbow point on the graph.



computed the average number of iterations. While they were all quite similar, both $t=4$ and $t=8$ had multiple trials where they did not converge before the max iteration was hit. For this reason I selected $t=2$, so that I can ensure the function converges every time.

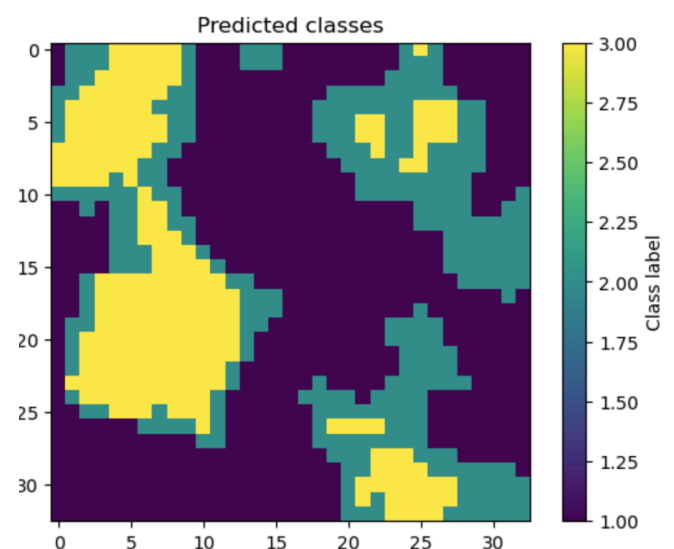
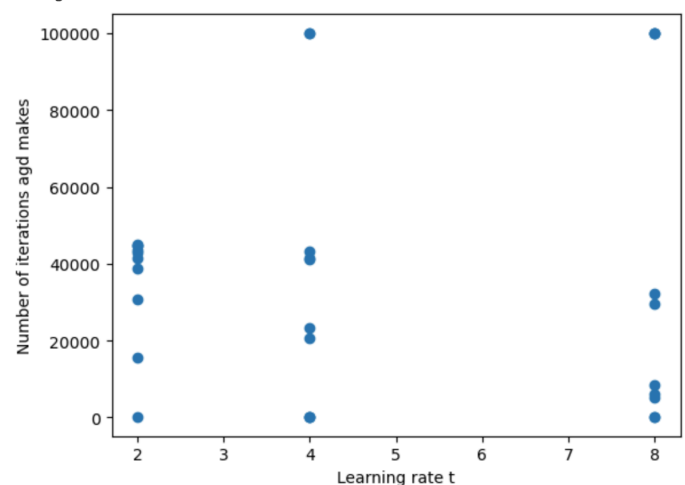
However, when applying this function to our data, the value of t that gave us an accurate classification seemed to change quite a bit depending on the initialisation of Θ_0 . When it did classify, I found that clustering the data before applying agd did lead to a much better classification than in q3, as it now classified into all 3 compounds.

Ideally, I think to improve this model, I would need to increase K , the number of clusters, in order to retain more detail in the data. This would also potentially help to retain a more constant optimal value of t . However when I tried increasing K , it did not work as effectively as it often returned overflow and runtime warnings.



To find t , I applied a similar process to q3. My graph showed that for the function to converge quickly and effectively, t between 2 and 8 would be optimal. I did more in depth analysis to decide between 2/4/8. I plotted the following graph using a for loop, which did 10 random trials of the function with different Θ_0 for each t , and then

Average number of iterations for $t=2$ = 34810.5
Average number of iterations for $t=4$ = 37027.2
Average number of iterations for $t=8$ = 38205.4



Question 6

$$L_0(\underline{\theta}) = -\frac{1}{N} \sum_{i=0}^{N-1} \sum_{k=0}^{M-1} y_{i,k} \log h_{\underline{\theta}_k}(\hat{x}_i)$$

$$\frac{\partial}{\partial \underline{\theta}_{j,k}} L_0(\underline{\theta}) = \frac{\partial}{\partial \underline{\theta}_{j,k}} \left(-\frac{1}{N} \sum_{i=0}^{N-1} \sum_{k=0}^{M-1} y_{i,k} \log h_{\underline{\theta}_k}(\hat{x}_i) \right)$$

$$= -\frac{1}{N} \sum_{i=0}^{N-1} \sum_{k=0}^{M-1} y_{i,k} \underbrace{\frac{\partial}{\partial \underline{\theta}_{j,k}} \log h_{\underline{\theta}_k}(\hat{x}_i)}_{\downarrow}$$

$$\frac{\partial}{\partial \underline{\theta}_{j,k}} \log h_{\underline{\theta}_k}(\hat{x}_i) = \frac{\partial}{\partial \underline{\theta}_{j,k}} \log \left(\frac{\exp(\langle \underline{x}_i, \underline{\theta}_k \rangle)}{\sum_{k'=0}^{M-1} \exp(\langle \underline{x}_i, \underline{\theta}_{k'} \rangle)} \right)$$

$$= \frac{\partial}{\partial \underline{\theta}_{j,k}} \left[\log(\exp(\langle \underline{x}_i, \underline{\theta}_k \rangle)) - \log \left(\sum_{k'=0}^{M-1} \exp(\langle \underline{x}_i, \underline{\theta}_{k'} \rangle) \right) \right]$$

$$= \frac{\partial}{\partial \underline{\theta}_{j,k}} \left(\langle \underline{x}_i, \underline{\theta}_k \rangle \right) - \frac{\partial}{\partial \underline{\theta}_{j,k}} \left(\log \sum_{k'=0}^{M-1} \exp(\langle \underline{x}_i, \underline{\theta}_{k'} \rangle) \right)$$

$$\frac{\partial}{\partial \underline{\theta}_{j,k}} (\langle \underline{x}_i, \underline{\theta}_k \rangle) = \frac{\partial}{\partial \underline{\theta}_{j,k}} \sum_{l=0}^{L-1} x_{i,l} \theta_{l,k}$$

$$\begin{aligned} & (= 0 \text{ when } l \neq j \\ & = x_{i,j} \text{ when } l = j) \end{aligned}$$

$$= x_{i,j}$$

$$\frac{\partial}{\partial \underline{\theta}_{j,k}} \left(\log \sum_{k'=0}^{M-1} \exp(\langle \underline{x}_i, \underline{\theta}_{k'} \rangle) \right) = \frac{\frac{\partial}{\partial \underline{\theta}_{j,k}} \exp(\langle \underline{x}_i, \underline{\theta}_k \rangle)}{\sum_{k'=0}^{M-1} \exp(\langle \underline{x}_i, \underline{\theta}_{k'} \rangle)}$$

(chain rule)

for the numerator we have: $\exp(\langle \underline{x}_i, \underline{\theta}_k \rangle) \cdot \frac{\partial}{\partial \underline{\theta}_{j,k}} \langle \underline{x}_i, \underline{\theta}_k \rangle$
again by chain rule. $= x_{i,j}$

hence, $= \frac{\exp(\langle \underline{x}_i, \underline{\theta}_k \rangle)}{\sum_{k'=0}^{M-1} \exp(\langle \underline{x}_i, \underline{\theta}_{k'} \rangle)} \cdot x_{i,j}$

$$= x_{i,j} h_{\underline{\theta}_k}(\hat{x}_i)$$

Combine:

$$\frac{\partial}{\partial \underline{\theta}_{j,k}} \log h_{\underline{\theta}_k}(\hat{x}_i) = x_{i,j} - x_{i,j} h_{\underline{\theta}_k}(\hat{x}_i) = x_{i,j} (1 - h_{\underline{\theta}_k}(\hat{x}_i))$$

Hence,

$$\frac{\partial}{\partial \theta_{j,k}} L_D(\theta) = -\frac{1}{N} \sum_{i=0}^{N-1} \sum_{k=0}^{M-1} y_{i,k} x_{i,j} (1 - h_{\theta_k}(\tilde{x}_i))$$

y_i is the ^{label} ~~production~~ vector for sample i .
i.e. a vector of 0s with a 1 in the k th column, where sample i is in k th class.

$$y_{i,k} = \begin{cases} 0 & \text{if sample } i \text{ not in class } k \\ 1 & \text{if sample } i \text{ in class } k. \end{cases}$$

So there are two cases, for when $k=c$ and $k \neq c$ where c is the correct class. OpenAI was used for the next step to understand how to combine both cases into one formula.

$$\begin{aligned} \frac{\partial}{\partial \theta_{j,k}} L_D(\theta) &= -\frac{1}{N} \sum_{i=0}^{N-1} x_{i,j} (y_{i,k} - h_{\theta_k}(\tilde{x}_i)) \\ &= \frac{1}{N} \sum_{i=0}^{N-1} x_{i,j} (h_{\theta_k}(\tilde{x}_i) - y_{i,k}) \end{aligned}$$

which is the j,k th component of $\nabla L_D(\theta)$

hence this is the componentwise form of

$$\nabla L_D(\theta) = \frac{1}{N} \sum_{i=0}^{N-1} \tilde{x}_i^T (h_{\theta}(\tilde{x}_i) - \underline{y}_i)$$

Question 7

assuming $L(\theta)$ is β -smooth.

Lemma 4.24 gives inequality:

$$L(\underline{\eta}) \leq L(\underline{\theta}) + \langle \nabla L(\underline{\theta}), \underline{\eta} - \underline{\theta} \rangle + \frac{\beta}{2} \|\underline{\eta} - \underline{\theta}\|_2^2$$

for a β -smooth function L . let $\underline{\eta} = \theta_{s+1}$ and $\underline{\theta} = \theta_s$ in our AGD function, we have:

$$\theta_{s+1} = \theta_s - \frac{G}{\sqrt{p} + \epsilon}$$

$$\text{let } = \theta_s - \frac{t}{p} G \quad \text{where } p = \sqrt{p} + \epsilon$$

$$\Rightarrow \theta_{s+1} - \theta_s = -\frac{t}{p} G$$

sub into inequality:

$$\begin{aligned} L(\theta_{s+1}) &\leq L(\theta_s) + \langle \nabla L(\theta_s), \theta_{s+1} - \theta_s \rangle + \frac{\beta}{2} \|\theta_{s+1} - \theta_s\|_2^2 \\ &= L(\theta_s) + \langle \nabla L(\theta_s), -\frac{t}{p} G \rangle + \frac{\beta}{2} \left\| -\frac{t}{p} G \right\|_2^2 \end{aligned}$$

($\nabla L(\theta_s) = G$ by definition)

$$\begin{aligned} &= L(\theta_s) + \langle G, -\frac{t}{p} G \rangle + \frac{\beta t^2}{2p^2} \|G\|_2^2 \\ &= L(\theta_s) - \frac{t}{p} \langle G, G \rangle + \frac{\beta t^2}{2p^2} \|G\|_2^2 \end{aligned}$$

(linearity in second slot)

$$\begin{aligned} &= L(\theta_s) - \frac{t}{p} \|G\|_2^2 + \frac{\beta t^2}{2p^2} \|G\|_2^2 \\ &= L(\theta_s) - \|G\|_2^2 \left(\frac{t}{p} - \frac{\beta t^2}{2p^2} \right) \end{aligned}$$

hence,

~~$$L(\theta_{s+1}) - L(\theta_s) = -\|G\|_2^2 \left(\frac{t}{p} - \frac{\beta t^2}{2p^2} \right)$$~~

~~difference in loss between iterations. Want to maximise this over t.~~

$$\|G\|_2^2 \left(\frac{t}{p} - \frac{\beta t^2}{2p^2} \right) \leq \underline{L(\theta_s) - L(\theta_{s+1})}$$

= decrease in loss from one iteration to the next. Want to maximise this over t.

Maximising :

$$\|G\|_2^2 \left(\frac{t}{p} - \frac{\beta t^2}{2p^2} \right)$$

$$\|G\|_2^2 > 0 \quad \text{constant.}$$

$$\frac{d}{dt} \left(\frac{t}{p} - \frac{\beta t^2}{2p^2} \right) = \frac{1}{p} - \frac{\beta t}{p^2} = 0 \Rightarrow t = \frac{p}{\beta}$$

$$\frac{d^2}{dt^2} \left(\frac{t}{p} - \frac{\beta t^2}{2p^2} \right) = -\frac{\beta}{p^2} < 0 \quad \text{so it is a maximum}$$

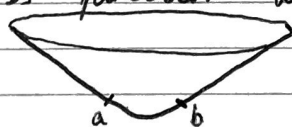
So our maximised t is : $t = \frac{\sqrt{v} + \epsilon}{\beta}$

for standard GD, lemma 4.25 gives us an optimal learning rate $\hat{t} = \frac{1}{\beta}$. So for AGD, our maximised learning rate has been scaled by $\sqrt{v} + \epsilon$.

$$V = \frac{1}{(n+1)m} \sum_{j=0}^n \sum_{k=0}^{m-1} G_{j,k}^2 \quad \text{is the mean of the squared gradients.}$$

So V gives us the average squared "steepness" of the loss function with parameter θ_s . This means when V is small, t will become smaller and when V is large t will be large. (as average gradient will reduce as you approach a minimum). This makes it more effective than GD as it will be less likely to jump over the minimum and diverge.

However, if the loss function took a shape similar to the following:



where outside point a and b the gradient is constant,

then GD may be more effective as t will remain constant until close to min, so AGD recomputing the average gradient every step will be unnecessarily slow.